AIML Pratical.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Connect

```
PART 6: REGRESSION (Manual implementations / variants)
"""

# ------------------------
# SECTION-1: IMPORTS & LOAD
# ------------------------
import warnings
warnings.filterwarnings("ignore")

import os
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
from sklearn.linear_model import LogisticRegression, LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor, GradientBoostingRegressor
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC

pd.options.display.max_columns = 200

# Attempt to load dataset robustly
DATA_PATH = "1_dataset.csv"
if not os.path.exists(DATA_PATH):
    # If running in Colab and user uploaded under a different name, try common alternates
    raise FileNotFoundError(f"{DATA_PATH} not found in working directory. Upload 1_dataset.csv to Colab.")

df = pd.read_csv(DATA_PATH, engine="python")
df.columns = df.columns.str.strip()

print("Loaded dataset:", DATA_PATH)
print("Shape:", df.shape)
print(df.head(3))
print(df.info())

# Mount drive (optional; safe to keep as in aiml_crop)
try:
    from google.colab import drive
    drive.mount('/content/drive')
df.columns = df.columns.str.strip()

print("Loaded dataset:", DATA_PATH)
print("Shape:", df.shape)
print(df.head(3))
print(df.info())

# Mount drive (optional; safe to keep as in aiml_crop)
try:
    from google.colab import drive
    drive.mount('/content/drive')
except Exception:
    pass
```

```
Loaded dataset: 1_dataset.csv
Shape: (78138, 6)
   Year  Month  Day  Hour  Temperature  Irradiance
0  2014      1    1     6         9.44        0.00
1  2014      1    1     7        11.87       99.09
2  2014      1    1     8        14.55      290.91
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 78138 entries, 0 to 78137
Data columns (total 6 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Year         78138 non-null  int64
 1   Month        78138 non-null  int64
 2   Day          78138 non-null  int64
 3   Hour         78138 non-null  int64
 4   Temperature  78138 non-null  float64
 5   Irradiance   78138 non-null  float64
dtypes: float64(2), int64(4)
memory usage: 3.6 MB
None
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

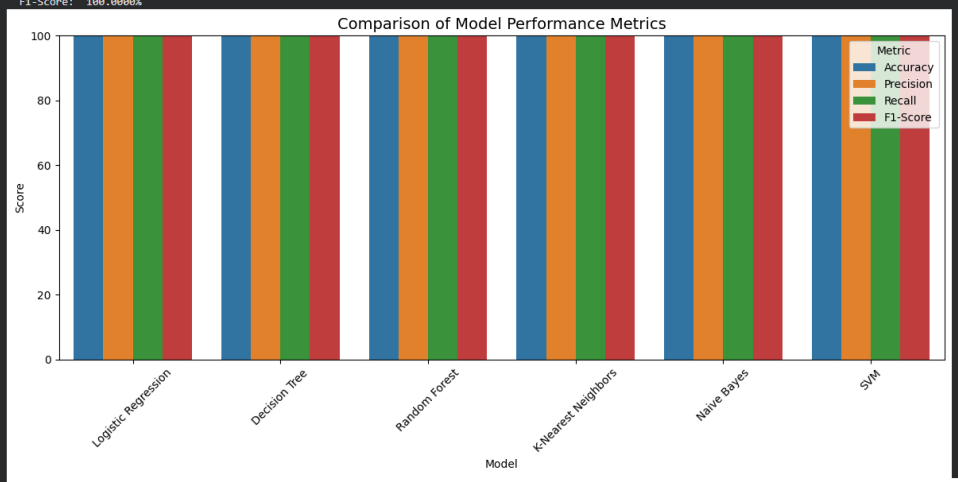
```
# ----------------------------------------
# SECTION-2: DATA CLEANING & FEATURE CREATION
# ----------------------------------------
"""## Section 2 - DATA CLEANING & FEATURE CREATION"""

# Helper: coerce likely numeric columns and impute
def coerce_numeric_and_impute(dataframe):
    dfc = dataframe.copy()
    # try to coerce common numeric-like columns
    for col in dfc.columns:
        if dfc[col].dtype == object:
            sample = dfc[col].dropna().astype(str).head(200)
            # if many entries look numeric, coerce
            if len(sample) > 0:
                numeric_like = sample.str.replace(r'[^\d\.\-]', '', regex=True).str.len().replace(0, np.nan).notna().sum()
                if numeric_like > len(sample) * 0.6:
                    dfc[col] = pd.to_numeric(dfc[col].astype(str).str.replace(',', '').str.strip(), errors='coerce')
    # fill numeric missing with median, categorical with mode (or 'missing')
    for col in dfc.columns:
        if pd.api.types.is_numeric_dtype(dfc[col]):
            median = dfc[col].median()
            dfc[col].fillna(median, inplace=True)
        else:
            if not dfc[col].mode().empty:
                dfc[col].fillna(dfc[col].mode().iloc[0], inplace=True)
            else:
                dfc[col].fillna("missing", inplace=True)
    return dfc

df = coerce_numeric_and_impute(df)

# Derived features inspired by aiml_crop.py
if {"Temperature", "Humidity"}.issubset(df.columns):
    df["Temp_Humidity"] = df["Temperature"] * df["Humidity"]
if {"Production", "Area"}.issubset(df.columns):
    df["Prod_per_Area"] = df["Production"].replace(0, np.nan) / df["Area"].replace(0, np.nan)
    df["Prod_per_Area"] = df["Prod_per_Area"].fillna(0)

# Convert object categorical cols to codes for manual algos
for c in df.select_dtypes(include=["object", "category"]).columns:
    df[c + "_code"] = df[c].astype("category").cat.codes

print("Feature Creation Completed")
print("Dataset shape after features:", df.shape)
print(df.select_dtypes(include=[np.number]).columns.tolist())
```

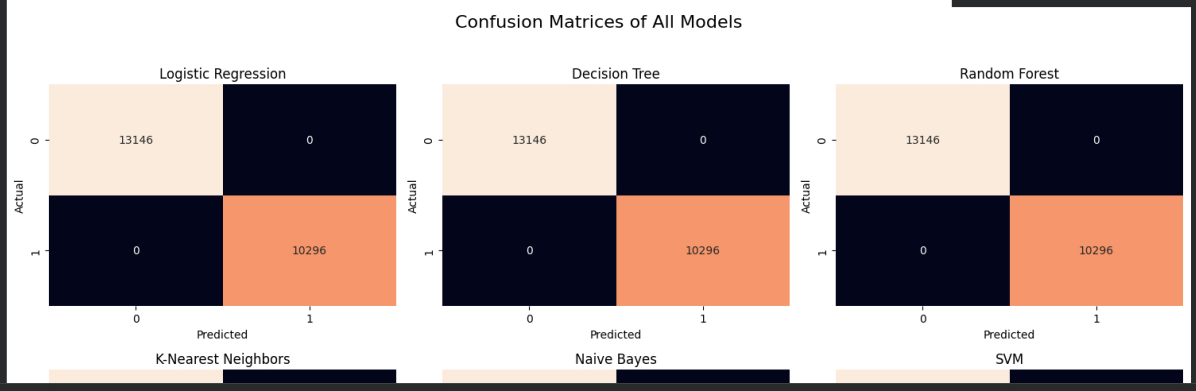
```
# Create target for classification similar to aiml_crop.py
if "Production" in df.columns:
    threshold = df["Production"].quantile(0.6)
    df["HighProduction"] = np.where(df["Production"] > threshold, 1, 0)
    target_col = "HighProduction"
else:
    # fallback to existing 'target' or create AutoTarget from first numeric column
    if "target" in df.columns:
        target_col = "target"
    else:
        numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
        if not numeric_cols:
            raise ValueError("No numeric columns available to build a target.")
        first_num = numeric_cols[0]
        df["AutoTarget"] = np.where(df[first_num] > df[first_num].median(), 1, 0)
        target_col = "AutoTarget"

print(f"Using target column for classification: {target_col}")
print(df[target_col].value_counts())
```

```
Feature Creation Completed
Dataset shape after features: (78138, 6)
['Year', 'Month', 'Day', 'Hour', 'Temperature', 'Irradiance']
Using target column for classification: AutoTarget
AutoTarget
0    43818
1    34320
Name: count, dtype: int64
```

```
# ------------------------------------------------------------
# PART 3 : MULTIPLE CLASSIFICATION MODELS COMPARISON (Using Libraries)
# ------------------------------------------------------------
"""## PART 3 : MULTIPLE CLASSIFICATION MODELS COMPARISON (Using Libraries)"""

# Select sensible features (mirror aiml_crop preferred features if available)
preferred = ["Temperature", "Humidity", "Soil_Moisture", "Area", "Crop_Year", "Temp_Humidity", "Prod_per_Area"]
features = [c for c in preferred if c in df.columns]
# If too few, supplement with top numeric by variance (excluding target)
if len(features) < 4:
    numeric_candidates = [c for c in df.select_dtypes(include=[np.number]).columns if c != target_col and c not in features]
    numeric_candidates_sorted = sorted(numeric_candidates, key=lambda x: df[x].var() if pd.api.types.is_numeric_dtype(df[x]) else 0, reverse=True)
    features += numeric_candidates_sorted[:max(0, 6 - len(features))]

print("Selected features for classification:", features)

X = df[features]
y = df[target_col].astype(int)

# Split and scale where needed
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y if len(np.unique(y))>1 else None)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

models_lib = {
    "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=50, random_state=42),
    "K-Nearest Neighbors": KNeighborsClassifier(n_neighbors=3),
    "Naive Bayes": GaussianNB(),
    "SVM": SVC(kernel='linear', probability=True, random_state=42)
}

print("MODELS ANALYSIS USING SKLEARN LIBRARIES")

results = []
y_preds = {}

for name, model in models_lib.items():
    try:
        if name in ["Logistic Regression", "K-Nearest Neighbors", "SVM"]:
            model.fit(X_train_scaled, y_train)
            y_pred = model.predict(X_test_scaled)
        else:
            model.fit(X_train, y_train)
            y_pred = model.predict(X_test)
    except Exception as e:
        print(f"{name} training failed: {e}")
        continue

    y_preds[name] = y_pred

    accuracy = accuracy_score(y_test, y_pred) * 100
    precision = precision_score(y_test, y_pred, zero_division=0) * 100
    recall = recall_score(y_test, y_pred, zero_division=0) * 100
    f1 = f1_score(y_test, y_pred, zero_division=0) * 100

    results.append({
        'Model': name,
        'Accuracy': accuracy,
        'Precision': precision,
        'Recall': recall,
        'F1-Score': f1
    })

    print(f"\n{name}:")
    print(f"  Accuracy:  {accuracy:.4f}%")
    print(f"  Precision: {precision:.4f}%")
    print(f"  Recall:    {recall:.4f}%")
    print(f"  F1-Score:  {f1:.4f}%")

# Plot metrics comparison
if results:
    metrics_df = pd.DataFrame(results)
    metrics_melted = metrics_df.melt(id_vars='Model', var_name='Metric', value_name='Score')
    plt.figure(figsize=(12, 6))
    sns.barplot(data=metrics_melted, x='Model', y='Score', hue='Metric')
    plt.title("Comparison of Model Performance Metrics", fontsize=14)
    plt.ylim(0, 100)
    plt.xticks(rotation=45)
    plt.legend(title='Metric')
    plt.tight_layout()
    plt.show()

    # Plot confusion matrices in grid
    n_models = len(y_preds)
    if n_models > 0:
        cols = 3
        rows = (n_models + cols - 1) // cols
        fig, axes = plt.subplots(rows, cols, figsize=(5 * cols, 4 * rows))
        axes = axes.flatten()
        for idx, (name, y_pred) in enumerate(y_preds.items()):
            cm = confusion_matrix(y_test, y_pred)
            sns.heatmap(cm, annot=True, fmt='d', cbar=False, ax=axes[idx])
            axes[idx].set_title(name)
            axes[idx].set_xlabel('Predicted')
            axes[idx].set_ylabel('Actual')
        # hide unused axes
        for ax in axes[len(y_preds):]:
            ax.axis('off')
        plt.suptitle('Confusion Matrices of All Models', fontsize=16)
        plt.tight_layout(rect=[0, 0, 1, 0.96])
        plt.show()
```

```
Selected features for classification: ['Temperature', 'Irradiance', 'Day', 'Hour', 'Month', 'Year']
MODELS ANALYSIS USING SKLEARN LIBRARIES

Logistic Regression:
  Accuracy:  100.0000%
  Precision: 100.0000%
  Recall:    100.0000%
  F1-Score:  100.0000%

Decision Tree:
  Accuracy:  100.0000%
  Precision: 100.0000%
  Recall:    100.0000%
  F1-Score:  100.0000%

Random Forest:
  Accuracy:  100.0000%
  Precision: 100.0000%
  Recall:    100.0000%
  F1-Score:  100.0000%

K-Nearest Neighbors:
  Accuracy:  99.9701%
  Precision: 99.9709%
  Recall:    99.9611%
  F1-Score:  99.9660%

Naive Bayes:
  Accuracy:  100.0000%
  Precision: 100.0000%
  Recall:    100.0000%
  F1-Score:  100.0000%

SVM:
  Accuracy:  100.0000%
  Precision: 100.0000%
  Recall:    100.0000%
  F1-Score:  100.0000%
```



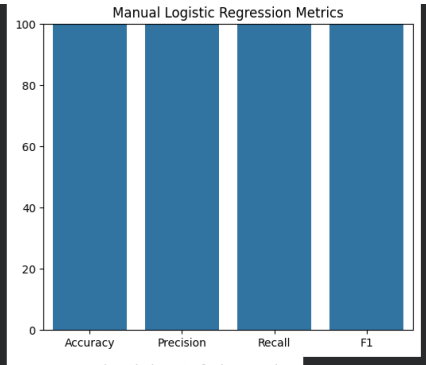


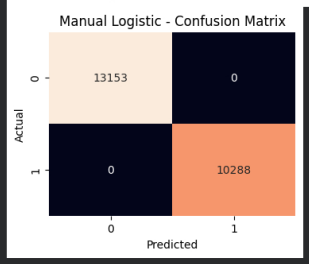

```python
# ------------------------------------------------------------
# PART 4 : MULTIPLE CLASSIFICATION MODELS COMPARISON (Without Using Libraries)
# ------------------------------------------------------------
"""## PART 4 : MULTIPLE CLASSIFICATION MODELS COMPARISON (Manual implementations)"""

# ---- Manual Logistic Regression ----
class ManualLogisticRegression:
    def __init__(self, learning_rate=0.01, n_iterations=1000):
        self.learning_rate = learning_rate
        self.n_iterations = n_iterations
        self.weights = None
        self.bias = None

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-np.clip(z, -250, 250)))

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0.0

        for _ in range(self.n_iterations):
            linear_model = np.dot(X, self.weights) + self.bias
            predictions = self.sigmoid(linear_model)

            dw = (1 / n_samples) * np.dot(X.T, (predictions - y))
            db = (1 / n_samples) * np.sum(predictions - y)

            self.weights -= self.learning_rate * dw
            self.bias -= self.learning_rate * db

    def predict(self, X):
        linear_model = np.dot(X, self.weights) + self.bias
        predictions = self.sigmoid(linear_model)
        return (predictions >= 0.5).astype(int)

def calc_metrics_manual(y_true, y_pred):
    tp = np.sum((y_true == 1) & (y_pred == 1))
    tn = np.sum((y_true == 0) & (y_pred == 0))
```

```
        fp = np.sum((y_true == 0) & (y_pred == 1))
        fn = np.sum((y_true == 1) & (y_pred == 0))
        total = tp + tn + fp + fn
        accuracy = (tp + tn) / total * 100 if total > 0 else 0
        precision = tp / (tp + fp) * 100 if (tp + fp) > 0 else 0
        recall = tp / (tp + fn) * 100 if (tp + fn) > 0 else 0
        f1 = 2 * (precision * recall) / (precision + recall) * 100 if (precision + recall) > 0 else 0
        cm = [[int(tn), int(fp)], [int(fn), int(tp)]]
        return accuracy, precision, recall, f1, cm

    # Prepare manual train/test split & scaling using full numeric feature set chosen earlier
    X_manual = X.values
    y_manual = y.values
    np.random.seed(42)
    indices = np.random.permutation(len(X_manual))
    test_size = int(0.3 * len(X_manual))
    test_idx = indices[:test_size]
    train_idx = indices[test_size:]

    X_train_m = X_manual[train_idx]
    X_test_m = X_manual[test_idx]
    y_train_m = y_manual[train_idx]
    y_test_m = y_manual[test_idx]

    mean_m = X_train_m.mean(axis=0)
    std_m = X_train_m.std(axis=0)
    std_m[std_m == 0] = 1.0
    X_train_scaled_m = (X_train_m - mean_m) / std_m
    X_test_scaled_m = (X_test_m - mean_m) / std_m

    # Train manual logistic
    mlr = ManualLogisticRegression(learning_rate=0.1, n_iterations=2000)
    mlr.fit(X_train_scaled_m, y_train_m)
    y_pred_mlr = mlr.predict(X_test_scaled_m)
    acc, prec, rec, f1, cm = calc_metrics_manual(y_test_m, y_pred_mlr)
    print("\nManual Logistic Regression -")
    print(f"Accuracy: {acc:.2f}% Precision: {prec:.2f}% Recall: {rec:.2f}% F1: {f1:.2f}%")
    plt.figure(figsize=(6,5))
    sns.barplot(x=["Accuracy","Precision","Recall","F1"], y=[acc,prec,rec,f1])
    plt.ylim(0,100)
    plt.title("Manual Logistic Regression Metrics")
    plt.show()
    plt.figure(figsize=(4,3))
    sns.heatmap(cm, annot=True, fmt='d', cbar=False)
    plt.title("Manual Logistic - Confusion Matrix")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.show()

    # ---- Manual Decision Tree (simple) ----
    class ManualDecisionTree:
        def __init__(self, max_depth=5, min_samples_split=2):
            self.max_depth = max_depth
            self.min_samples_split = min_samples_split
            self.tree = None

        def _gini(self, y):
            if len(y) == 0:
                return 0
            p1 = np.sum(y) / len(y)
            p0 = 1 - p1
            return 1 - (p0**2 + p1**2)

        def _best_split(self, X, y):
            best_gini = float('inf')
            best_feat = None
            best_thresh = None
            n_features = X.shape[1]
            for feat in range(n_features):
                values = np.unique(X[:, feat])
                for thr in values:
                    left = y[X[:, feat] <= thr]
                    right = y[X[:, feat] > thr]
                    if len(left) == 0 or len(right) == 0:
                        continue
                    gini = (len(left) * self._gini(left) + len(right) * self._gini(right)) / len(y)
                    if gini < best_gini:
                        best_gini = gini
                        best_feat = feat
                        best_thresh = thr
            return best_feat, best_thresh

        def _build(self, X, y, depth=0):
            if depth >= self.max_depth or len(y) < self.min_samples_split or len(np.unique(y)) == 1:
                return float(np.round(np.mean(y)))
            feat, thr = self._best_split(X, y)
            if feat is None:
                return float(np.round(np.mean(y)))
            left_mask = X[:, feat] <= thr
            left = self._build(X[left_mask], y[left_mask], depth + 1)
            right = self._build(X[~left_mask], y[~left_mask], depth + 1)
            return {"feature": feat, "threshold": thr, "left": left, "right": right}

        def fit(self, X, y):
            self.tree = self._build(X, y, 0)

        def _predict_one(self, x, node):
            if not isinstance(node, dict):
                return int(node)
            if x[node["feature"]] <= node["threshold"]:
                return self._predict_one(x, node["left"])
            else:
                return self._predict_one(x, node["right"])

        def predict(self, X):
            return np.array([self._predict_one(x, self.tree) for x in X])

    mdt = ManualDecisionTree(max_depth=6)
    mdt.fit(X_train_m, y_train_m)
    y_pred_mdt = mdt.predict(X_test_m)
    acc_dt, prec_dt, rec_dt, f1_dt, cm_dt = calc_metrics_manual(y_test_m, y_pred_mdt)
    print("\nManual Decision Tree -")
    print(f"Accuracy: {acc_dt:.2f}% Precision: {prec_dt:.2f}% Recall: {rec_dt:.2f}% F1: {f1_dt:.2f}%")

    # ---- Manual Random Forest (ensemble of manual DTs) ----
    class ManualRandomForest:
        def __init__(self, n_estimators=10, max_depth=5):
            self.n_estimators = n_estimators
            self.max_depth = max_depth
            self.trees = []
            self.feat_indices = []

        def _bootstrap(self, X, y):
            idx = np.random.choice(len(X), len(X), replace=True)
            return X[idx], y[idx]

        def fit(self, X, y):
            n_features = X.shape[1]
            m = int(np.sqrt(n_features)) if n_features > 1 else 1
            self.trees = []
            self.feat_indices = []
            for _ in range(self.n_estimators):
                Xs, ys = self._bootstrap(X, y)
```

```
                feat_idx = np.random.choice(n_features, m, replace=False)
                tree = ManualDecisionTree(max_depth=self.max_depth)
                tree.fit(Xs[:, feat_idx], ys)
                self.trees.append(tree)
                self.feat_indices.append(feat_idx)

        def predict(self, X):
            preds = []
            for tree, fi in zip(self.trees, self.feat_indices):
                preds.append(tree.predict(X[:, fi]))
            preds = np.array(preds)
            # majority vote
            maj = np.round(np.mean(preds, axis=0)).astype(int)
            return maj

    mrf = ManualRandomForest(n_estimators=8, max_depth=6)
    mrf.fit(X_train_m, y_train_m)
    y_pred_mrf = mrf.predict(X_test_m)
    acc_rf, prec_rf, rec_rf, f1_rf, cm_rf = calc_metrics_manual(y_test_m, y_pred_mrf)
    print("\nManual Random Forest -")
    print(f"Accuracy: {acc_rf:.2f}% Precision: {prec_rf:.2f}% Recall: {rec_rf:.2f}% F1: {f1_rf:.2f}%")

    # ---- Manual KNN ----
    class ManualKNN:
        def __init__(self, k=5):
            self.k = k

        def fit(self, X, y):
            self.X_train = X
            self.y_train = y

        def _dist(self, a, b):
            return np.sqrt(np.sum((a - b) ** 2))

        def predict(self, Xq):
            preds = []
            for q in Xq:
                dists = np.array([self._dist(q, t) for t in self.X_train])
                idx = np.argsort(dists)[:self.k]
                labels = self.y_train[idx]
                pred = np.argmax(np.bincount(labels))
                preds.append(pred)
            return np.array(preds)

    mknn = ManualKNN(k=5)
    mknn.fit(X_train_m, y_train_m)
    y_pred_mknn = mknn.predict(X_test_m)
    acc_knn, prec_knn, rec_knn, f1_knn, cm_knn = calc_metrics_manual(y_test_m, y_pred_mknn)
    print("\nManual KNN -")
    print(f"Accuracy: {acc_knn:.2f}% Precision: {prec_knn:.2f}% Recall: {rec_knn:.2f}% F1: {f1_knn:.2f}%")

    # ---- Manual Naive Bayes ----
    class ManualNaiveBayes:
        def fit(self, X, y):
            X = X if isinstance(X, np.ndarray) else X.values
            y = y if isinstance(y, np.ndarray) else y
            self.classes = np.unique(y)
            self.means = {}
            self.vars = {}
            self.priors = {}
            for c in self.classes:
                Xc = X[y == c]
                self.means[c] = Xc.mean(axis=0)
                self.vars[c] = Xc.var(axis=0) + 1e-6
                self.priors[c] = len(Xc) / len(X)

        def _pdf(self, class_idx, x):
            mean = self.means[class_idx]
            var = self.vars[class_idx]
            numerator = np.exp(- ((x - mean) ** 2) / (2 * var))
            denominator = np.sqrt(2 * np.pi * var)
            return numerator / denominator

        def _predict_one(self, x):
            post = {}
            for c in self.classes:
                prior = np.log(self.priors[c])
                likelihood = np.sum(np.log(self._pdf(c, x) + 1e-12))
                post[c] = prior + likelihood
            return max(post, key=post.get)

        def predict(self, X):
            X = X if isinstance(X, np.ndarray) else X
            return np.array([self._predict_one(x) for x in X])

    mnb = ManualNaiveBayes()
    mnb.fit(X_train_m, y_train_m)
    y_pred_mnb = mnb.predict(X_test_m)
    acc_nb, prec_nb, rec_nb, f1_nb, cm_nb = calc_metrics_manual(y_test_m, y_pred_mnb)
    print("\nManual Naive Bayes -")
    print(f"Accuracy: {acc_nb:.2f}% Precision: {prec_nb:.2f}% Recall: {rec_nb:.2f}% F1: {f1_nb:.2f}%")

    # ---- Manual SVM (linear hinge using SGD) ----
    class ManualSVM:
        def __init__(self, learning_rate=0.001, lambda_param=0.01, n_iterations=1000):
            self.lr = learning_rate
            self.lam = lambda_param
            self.n_iter = n_iterations
            self.w = None
            self.b = 0

        def fit(self, X, y):
            X = X if isinstance(X, np.ndarray) else X
            y = np.where(y == 0, -1, 1)
            n_samples, n_features = X.shape
            self.w = np.zeros(n_features)
            for _ in range(self.n_iter):
                for idx, x_i in enumerate(X):
                    condition = y[idx] * (np.dot(x_i, self.w) + self.b)
                    if condition >= 1:
                        dw = 2 * self.lam * self.w
                        db = 0
                    else:
                        dw = 2 * self.lam * self.w - (y[idx] * x_i)
                        db = -y[idx]
                    self.w -= self.lr * dw
                    self.b -= self.lr * db

        def predict(self, X):
            X = X if isinstance(X, np.ndarray) else X
            approx = np.dot(X, self.w) + self.b
            return np.where(approx >= 0, 1, 0)

    msvm = ManualSVM(learning_rate=0.001, lambda_param=0.01, n_iterations=1500)
    msvm.fit(X_train_m, y_train_m)
    y_pred_msvm = msvm.predict(X_test_m)
    acc_svm, prec_svm, rec_svm, f1_svm, cm_svm = calc_metrics_manual(y_test_m, y_pred_msvm)
    print("\nManual SVM -")
    print(f"Accuracy: {acc_svm:.2f}% Precision: {prec_svm:.2f}% Recall: {rec_svm:.2f}% F1: {f1_svm:.2f}%")
```

```
Manual Logistic Regression -
Accuracy: 100.00% Precision: 100.00% Recall: 100.00% F1: 10000.00%
```






```
Manual Decision Tree -
Accuracy: 100.00% Precision: 100.00% Recall: 100.00% F1: 10000.00%

Manual Random Forest -
Accuracy: 56.11% Precision: 0.00% Recall: 0.00% F1: 0.00%
```

[ ]

```python
# ------------------------------------------------------------
# PART 5 : REGRESSION (Using Libraries)
# ------------------------------------------------------------
"""## PART 5 : MULTIPLE REGRESSION MODELS COMPARISON (Using Libraries)"""

if "Production" in df.columns:
    Xr = df[[c for c in df.select_dtypes(include=[np.number]).columns if c != "Production"]]
    yr = df["Production"]
    Xr_train, Xr_test, yr_train, yr_test = train_test_split(Xr, yr, test_size=0.2, random_state=42)
    scaler_r = StandardScaler()
    Xr_train_s = scaler_r.fit_transform(Xr_train)
    Xr_test_s = scaler_r.transform(Xr_test)

    reg_models = {
        "Linear Regression": LinearRegression(),
        "Ridge": Ridge(alpha=1.0),
        "Lasso": Lasso(alpha=0.1, max_iter=5000),
        "ElasticNet": ElasticNet(alpha=0.1, l1_ratio=0.5, max_iter=5000),
        "Random Forest": RandomForestRegressor(n_estimators=100, random_state=42)
    }

    reg_results = []
    for name, model in reg_models.items():
        try:
            if name in ["Linear Regression", "Ridge", "Lasso", "ElasticNet"]:
                model.fit(Xr_train_s, yr_train)
                yr_pred = model.predict(Xr_test_s)
            else:
                model.fit(Xr_train, yr_train)
                yr_pred = model.predict(Xr_test)
        except Exception as e:
            print(f"Regression model {name} failed: {e}")
            continue
        mae = np.mean(np.abs(yr_test - yr_pred))
        rmse = np.sqrt(np.mean((yr_test - yr_pred) ** 2))
        r2 = 1 - np.sum((yr_test - yr_pred) ** 2) / (np.sum((yr_test - np.mean(yr_test)) ** 2) + 1e-12)
        reg_results.append({"Model": name, "MAE": round(mae, 3), "RMSE": round(rmse, 3), "R2": round(r2, 4)})
        print(f"{name}: MAE={mae:.3f} RMSE={rmse:.3f} R2={r2:.4f}")

    if reg_results:
        res_df = pd.DataFrame(reg_results)
        print("\n--- Regression Model Comparison ---")
        print(res_df.to_string(index=False))
        plt.figure(figsize=(9,4))
        plt.bar(res_df["Model"], res_df["R2"])
        plt.xlabel("Model")
        plt.ylabel("R2 Score")
        plt.title("Regression Model Performance Comparison")
        plt.xticks(rotation=45)
        plt.grid(axis='y', linestyle='--', alpha=0.5)
        plt.tight_layout()
        plt.show()
else:
    print("\nNo 'Production' column found; skipping regression comparison using libraries.")
```

[ ]

```python
# ------------------------------------------------------------
# PART 6 : MULTIPLE REGRESSION MODELS COMPARISON (Manual implementations)
# ------------------------------------------------------------
"""## PART 6 : MULTIPLE REGRESSION MODELS COMPARISON (Manual implementations)"""

# Provide a compact manual linear regression (gradient descent) and a manual KNN regressor if Production exists
if "Production" in df.columns:
    # Manual Linear Regression (GD)
    df_r = df.copy()
    cols_r = [c for c in df_r.select_dtypes(include=[np.number]).columns if c != "Production"]
    if len(cols_r) > 0:
        X_lr = df_r[cols_r].values
        y_lr = df_r["Production"].values.reshape(-1,1)
        np.random.seed(42)
        idx = np.random.permutation(len(X_lr))
        s = int(0.75 * len(X_lr))
        X_train_lr, X_test_lr = X_lr[idx[:s]], X_lr[idx[s:]]
        y_train_lr, y_test_lr = y_lr[idx[:s]], y_lr[idx[s:]]

        # scale
        mean_lr = X_train_lr.mean(0)
        sd_lr = X_train_lr.std(0)
        sd_lr[sd_lr == 0] = 1
        X_train_lr_s = (X_train_lr - mean_lr)/sd_lr
        X_test_lr_s = (X_test_lr - mean_lr)/sd_lr

        class LinearGD:
            def __init__(self, lr=0.01, epochs=2000):
                self.lr = lr
                self.epochs = epochs
```

```
            def fit(self, X, y):
                m, n = X.shape
                self.W = np.zeros((n,1))
                self.b = 0.0
                for _ in range(self.epochs):
                    pred = X @ self.W + self.b
                    dW = (X.T @ (pred - y)) / m
                    dB = np.sum(pred - y) / m
                    self.W -= self.lr * dW
                    self.b -= self.lr * dB

            def predict(self, X):
                return X @ self.W + self.b

        lin = LinearGD(lr=0.01, epochs=3000)
        lin.fit(X_train_lr_s, y_train_lr)
        pred_lr = lin.predict(X_test_lr_s)
        mae_lr = np.mean(np.abs(y_test_lr - pred_lr))
        rmse_lr = np.sqrt(np.mean((y_test_lr - pred_lr)**2))
        r2_lr = 1 - np.sum((y_test_lr - pred_lr)**2) / (np.sum((y_test_lr - np.mean(y_test_lr))**2) + 1e-12)
        print("\nManual Linear Regression:")
        print("MAE:", mae_lr, "RMSE:", rmse_lr, "R2:", r2_lr)
        plt.figure(figsize=(6,5))
        plt.scatter(y_test_lr, pred_lr, alpha=0.6)
        mn = min(y_test_lr.min(), pred_lr.min())
        mx = max(y_test_lr.max(), pred_lr.max())
        plt.plot([mn,mx],[mn,mx],'r--')
        plt.xlabel("Actual Production")
        plt.ylabel("Predicted Production")
        plt.title("Actual vs Predicted — Manual Linear Regression")
        plt.grid(True)
        plt.show()

    # Manual KNN Regression
    X_knn = X_lr
    y_knn = y_lr
    np.random.seed(42)
    idx = np.random.permutation(len(X_knn))
    s = int(0.75 * len(X_knn))
    X_train_knn, X_test_knn = X_knn[idx[:s]], X_knn[idx[s:]]
    y_train_knn, y_test_knn = y_knn[idx[:s]], y_knn[idx[s:]]
    # normalize
    m_knn = X_train_knn.mean(0)
    sd_knn = X_train_knn.std(0)
    sd_knn[sd_knn == 0] = 1
    X_train_knn_s = (X_train_knn - m_knn) / sd_knn
    X_test_knn_s = (X_test_knn - m_knn) / sd_knn

    class KNNReg:
        def __init__(self, k=3):
            self.k = k
        def fit(self, X, y):
            self.X = X
            self.y = y
        def predict(self, Xq):
            preds = []
            for q in Xq:
                d = np.sqrt(np.sum((self.X - q)**2, axis=1))
                idx = np.argsort(d)[:self.k]
                preds.append(np.mean(self.y[idx]))
            return np.array(preds).reshape(-1,1)

    knr = KNNReg(k=3)
    knr.fit(X_train_knn_s, y_train_knn)
    pred_knn = knr.predict(X_test_knn_s)
    mae_knn = np.mean(np.abs(y_test_knn - pred_knn))
    rmse_knn = np.sqrt(np.mean((y_test_knn - pred_knn)**2))
    r2_knn = 1 - np.sum((y_test_knn - pred_knn)**2) / (np.sum((y_test_knn - np.mean(y_test_knn))**2) + 1e-12)
    print("\nManual KNN Regression:")
    print("MAE:", mae_knn, "RMSE:", rmse_knn, "R2:", r2_knn)
    plt.figure(figsize=(6,5))
    plt.scatter(y_test_knn, pred_knn, alpha=0.6)
    mn = min(y_test_knn.min(), pred_knn.min())
    mx = max(y_test_knn.max(), pred_knn.max())
    plt.plot([mn,mx],[mn,mx],'r--')
    plt.xlabel("Actual Production")
    plt.ylabel("Predicted Production")
    plt.title("Actual vs Predicted — Manual KNN Regression")
    plt.grid(True)
    plt.show()
else:
    print("\nSkipping manual regression blocks because 'Production' column is absent.")
```